

# SORTNET

## FASE III

Simon Holm, Johannes Hesselager Rothe & Anne Sofie Nielsen Høj

sihol24, jorot24 & anho024

Fasekoordinator: Anne Sofie Nielsen Høj

DM574 - gruppe 15

20. December 2024

# Indhold

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>Introduktion</b>                  | <b>3</b>  |
| <b>2</b> | <b>Metode</b>                        | <b>3</b>  |
| 2.1      | Brainstorming . . . . .              | 3         |
| 2.2      | Vurdering . . . . .                  | 3         |
| <b>3</b> | <b>Programopbygning</b>              | <b>4</b>  |
| 3.1      | Programmeringsparadimer . . . . .    | 4         |
| 3.2      | Hjælpefunktioner . . . . .           | 4         |
| 3.2.1    | identical_sorted_list . . . . .      | 4         |
| 3.2.2    | prune_below . . . . .                | 5         |
| 3.2.3    | sort_by_score . . . . .              | 5         |
| 3.2.4    | keep_x_filters . . . . .             | 5         |
| 3.2.5    | keep_percentage_filters . . . . .    | 5         |
| <b>4</b> | <b>Programmer</b>                    | <b>6</b>  |
| 4.1      | first_prune . . . . .                | 6         |
| 4.2      | second_prune . . . . .               | 6         |
| 4.3      | third_prune . . . . .                | 7         |
| <b>5</b> | <b>Diskussion</b>                    | <b>9</b>  |
| 5.1      | Sammenligning af køretid . . . . .   | 9         |
| 5.2      | Scoringsystem . . . . .              | 9         |
| 5.3      | Optimering af second_prune . . . . . | 9         |
| 5.4      | Kombinering af prunes . . . . .      | 10        |
| 5.5      | Hvor meget skal prunes? . . . . .    | 11        |
| <b>6</b> | <b>Konklusion</b>                    | <b>12</b> |
| <b>7</b> | <b>Bilag</b>                         | <b>13</b> |
| 7.1      | prune.py . . . . .                   | 13        |
| <b>8</b> | <b>corrections for slides</b>        | <b>16</b> |

# 1 Introduktion

Denne rapport omhandler tredje og afsluttende fase af sorteringsnetværk projektet. Projektet er et afsluttende gruppeprojekt i faget introduktion til programmering. I projektet anvendes Python som programmeringssprog.

Dette projekt omhandler implementering af forskellige prune funktioner, så *network\_finder* kan køre med flere kanaler.

I de efterfølgende afsnit vil de forskellige versioner af prune implementeringen samt hjælpefunktionerne blive gennemgået, inklusiv forklaring samt visuelt med kode eksempler.

## 2 Metode

Denne del omhandler de metoder, der er blevet brugt til at udvikle de forskellige algoritmer for dette projekt. Modsat *boring-prune*, hvor metoden blev udleveret fra starten, er disse implementeringer af prune såsom *second-prune* og *third-prune* blevet udviklet af gruppen selv.

De mest to bemærkelsesværdige elementer af dette projekt inkluderede brainstorming i forhold til udvikling af prune modellerne samt vurdering af hvilken der er mest optimal af disse modeller.

### 2.1 Brainstorming

I skabningen af prune algoritmerne, brainstormede gruppen i fællesskab flere ideer til prune algoritmer. Gruppen blev hurtigt enig i, måder hvorpå algoritmer kunne scores, i forhold til hvor tæt de kunne være fra et sorteringsnetværk, hvorefter de ville kunne udlede flere mulige algoritme strukturer. Efterfølgende blev der brugt nogle redskaber til at lave en visuel formidling af sorteringsnetværk, sub-lister og comparatorer, med ideen om at prøve at finde måder at score dem på.

### 2.2 Vurdering

Ligeledes har optimering samt vurdering været en stor del af projektet, her inkluderes hvorvidt en given prune algoritme fjerner filtre. Der er meget plads til at optimere hastigheden af prune, hvor der optimeres hvilke filtre den pruner. Denne optimering og vurdering har primært været gjort med 2 vurderingskrav:

1. Finder algoritmen et optimalt sorteringsnetværk?
2. Hvad er hastigheden af algoritmen?

Da det samlede projekt, inklusive fase 1 og 2, handler om at finde det mest optimale sorteringsnetværk muligt, tog gruppen beslutningen om at ignorere algoritmer, der ikke resulterede i et sorteringsnetværk af optimal længde, ikke ville have været vurderet til at have løst opgaven. Dette er gjort bevidst, og vil blive gjort klart i løbet af rapporten. Visse valg kunne være taget for at drastisk reducere tiden det ville tage en prune-algoritme, at finde et sorteringsnetværk, men den ville også lede til at der ville være flere comparatorer end nødvendigt. For at vurdere om at et vis antal kanaler er optimalt, blev denne artikel brugt[1].

Hvis en prune-algoritme udleder et optimalt sorteringsnetværk, er det efterfølgende essentielt, at få køretiden for den algoritme så langt ned som muligt. Da en prune-algoritmen skal kunne håndtere eksponentielt flere filtre med højere antal kanaler, vil en reduktion på bare et par millisekunder pr. filter, blive til lang tid hvis der skal håndteres f.eks. en 100.000 filtre. Denne optimering blev gjort, ved at gruppen gennemgik diverse algoritmer og undersøgte om der blev lavet f.eks. unødvendig brug af andre funktioner, truth-checking, samt at undersøge andre optimeringsmetoder der vil blive diskuteret senere i rapporten.

## 3 Programopbygning

Denne sidste fase indeholder kun et modul, *prune.py*, hvori der er seks funktioner, der er tre forskellige implementeringer af *prune.py* samt fem hjælpefunktioner.

Disse moduler er baseret omkring tidligere afleveret modul *network\_finder* og udleveret python fil *prune.py*

### 3.1 Programmeringsparadimer

I dette projekt bruges der primært funktionel og imperativ programmering. Dette tager udgangspunkt i gruppens større kendskab til netop disse programmeringsparadigmer. Gruppen blev enige om, at hvis et specifikt problem ville kunne løses markant bedre med rekursiv programmering, ville programmets funktionalitet vægtes højere og derved ville rekursiv programmering bruges, men dette scenarie opleves ikke.

Der har været en del fokus på at bruge de funktioner, som f.eks. bruges i forbindelse med forelæsningsne og øvelsestimer for derved at undgå funktioner og metoder uden for gruppens forståelse. Vise indbyggede Python funktioner kunne muligvis gøre programmet hurtigere, men ville kræve, at gruppen skulle bruge ting, de ikke forstod præcist hvordan virkede.

While-løkker har været hyppigt brugt, af den primære baggrund af deres læselighed. Gruppen har vægtet at programmet er overskueligt og velstruktureret, og derved har while-løkker været et vigtigt redskab for f.eks. at kunne gå igennem hvert element i en subliste, i outputtet af et filter.

### 3.2 Hjælpefunktioner

Gruppen har diskuteret brugen af hjælpefunktioner ved projektets begyndelse. Der blev gruppen enige om det blev vurderet, hvis en funktion var meget lang, eller det ville forbedre funktions læselighed med en hjælpefunktion, ville det blive taget op i gruppen.

I dette modul er der dog blevet implementeret flere essentielle hjælpefunktioner. Disse funktioner hjælper til at forkorte de enkle funktioner.

Her vil de forskellige af implementering blev gennemgået med forklaring samt visuelt med kode eksempler.

#### 3.2.1 identical\_sorted\_list

*Identical\_sorted\_list* får en liste af lister af 0'er og 1'taller på længde  $n$  som input, og returnerer en sorteret identisk liste. Dette kan gøres, da antallet af 1'er er kendt. Ved f.eks. listen  $[0, 1, 1, 0]$  kan der udledes at den sorterede liste kopi skal indeholde to 0'er og to 1'taller. Derfor vil den identiske endelige sorterede liste være  $[0, 0, 1, 1]$ .

Funktionen bruger her en while-løkke, hvor den summere elementerne i output, altså de tilsvarende mængder af 1'taller i listen, til at bygge en liste som sorteret identisk liste. Da mængden af 1-taller kendes, skal de først indsættes, derefter indsættes mængden af 1-taller, 0-taller. Dette gentages for alle sub-listerne i output, indtil der er opbygget en identisk sorteret liste.

### 3.2.2 `prune_below`

*Prune\_below* funktionen er endnu en hjælpefunktion til *prune* funktionerne. Den anvendes både til *second\_prune* og *third\_prune*. Funktionen tager en liste af lister med filtre og en liste med scoren som input.

Funktionens formål er, at *prune* alle filtre hvis score ligger under gennemsnittet. Dette gøres ved at tage et gennemsnit af listen af scores, hvorefter de sammenlignes i listen med nye scores. Hvis en score ligger over gennemsnittet gemmes det tilsvarende filter i en ny liste. Den nye liste returneres til sidst.

Hvis *prune\_below* får en filter liste:  $[f1, f2, f3, f4, f5, f6, f7]$ , og en score liste:  $[1, 5, 7, 8, 6, 5, 3]$ . Vil den første regne gennemsnittet ud til 5, og returnerer de filtre der har en score der er det samme eller mindre end gennemsnittet. I dette tilfælde vil det være  $[f1, f2, f6, f7]$ .

### 3.2.3 `sort_by_score`

Funktionen *sort\_by\_score*'s formål er at sortere en liste af filtere efter deres score. Algoritmen bruger en *while*-løkke til at gennemgå listen i forhold de scorene. Hvis et element på plads 1, er større end et element på plads 2, bliver scorene og filtrene i deres respektive lister byttet om. Derefter bliver kontrolvariablen sat til en lavere værdi, givet scoren ikke er nul, i tilfælde der ikke gøres noget. Algoritmen er lavet som en udelukkende bivirknings algoritme, da den ikke skal ændre noget om scorene eller filtrene, men sørge for at listerne er sorteret.

For eksempel kan en *prune*funktion lave scorelisten  $[5, 2, 1, 2, 1, 3, 4]$  til 7 filtre  $[f1, f2, f3, f4, f5, f6, f7]$ , der hver i sær har fået tildelt en score. *sort\_by\_score*-funktionen vil sortere listen og samtidig sortere de 7 filtre efter den tilsvarende score. Scorelisten vil sorteres til  $[1, 1, 2, 2, 3, 4, 5]$  og listen med filtre vil sorteres som følgende:  $[f3, f5, f2, f4, f6, f7, f1]$

### 3.2.4 `keep_x_filters`

*Keep\_x\_filters* returnerer  $x$  antal af de bedste filters. Så hvis der vurderes, efter hver *extend*, at algoritmen kun skal beholde de 20 bedste filtre, baseret på scoringsystemet i *prune* algoritmerne, så vil *keep\_x\_filters* kun returnere de 20 laveste scores, da *prune* algoritmerne er lavet sådan, at jo lavere score desto bedre. I tilfældet af der ikke er 20 filtere, men i stedet kun 16, vil algoritmen returnere alle de filtre der er, så i dette tilfælde 16.

### 3.2.5 `keep_percentage_filters`

*Keep\_percentage\_filters* er en nærmest identisk kopi af *keep\_x\_filters*, bortset fra, at den vælger en bestemt mængde filtre der skal beholdes, derfra vælges der en procent mængde af de bedste filtere der beholdes. F.eks. kunne det være smart at beholde de 20% bedste filtere.

## 4 Programmer

### 4.1 first\_prune

Den første prune implementering er baseret på ideen, om at fjerne dubletter ved hjælp af en boolsk værdi *duplicates*, som fortæller hvorvidt en list af filter har en dublet.

Her bruges to while-løkker, hvor *i* er den første kontrol variable sat til 1. I den inderste while-løkke kører kontrolvariablen *j*. Både i den indre og ydre while-løkke kontrolleres der hvorvidt listen indeholder dubletter. Her har den yderste har en **false** værdi og den inderste har en **true** værdi.

Her ses hvorvidt filtre er dubletter af hinanden. Eksempelvis hvis funktionen får 5 filtre som input, filter [1, 2, 3, 4, 5] så vil det første filter blive puttet i resultat-listen. Efterfølgende vil det næste filter blive sammenlignet med filtre elementer allerede i resultat-listen og hvis det ikke eksisterer i listen vil det blive puttet for enden af listen. Den proces gentages for hvert filter i input-listen.

I dette eksempel vil 1 samt 2 bliver tilføjet til den nye liste uden problemer, da de ikke er dubletter af hinanden, hvorefter funktionen tjekker nummer 3. Her ses det, at element 3, er en dublet af filter nr. 1, dette filter bliver derfor ikke tilføjet, da funktionen siger *true*. Så kører den videre til det næste element hvor den bliver *false* igen og dermed bliver det næste element, 4, tilføjet. På denne måde køres der igennem hele listen, til sidst vil der være en liste med [1,2,4,5]. Da de andre elementer ikke allerede eksisterer i listen. Dermed sørger funktionen for at, der ikke er nogle dubletter. Som 1 og 3, som havde det samme filter og dermed ville give det samme output. Til sidst vil der være en liste med [1, 2, 4, 5]

Da de individuelle filter sorteres, kan der ses hvilke output de ville have. Hvis filtrene er ens, havde de samme muligheder og ville dermed de have samme output. Dermed var ideen at hvis de filtre der er dubletter fjernes, laves derigennem det mest optimale sorteringsnetværk, ved at de bliver kørt igennem de filtre på den mest optimale måde.

### 4.2 second\_prune

*Second\_prune* tager udgangspunkt i et scoringssystem. Ved at give hvert filter en score, kan *identical\_sorted\_list*, filtrerer filtrene efter deres givne score. Således kan funktionen sortere ikke relevante filtre fra listen af filtre og derved optimere programmet.

Funktionen skal kunne bestemme hvor sorteret et filter er. Det kan gøres ved at sammenligne et output i filteret med en sorteret kopi af den samme liste. Dette gøres af funktionen *identical\_sorted\_list*. Grunden til at hvert filter bliver sammenlignet med en identisk sorteret liste, er for at kunne estimere hvor mange comparatorers et filter mangler. Ved f.eks. sub-listen [0, 1, 0, 1] vil den blive sammenlignet med sin identiske sortrede liste [0, 0, 1, 1], den vil 2 uoverensstemmelser, generelt set vil der altid være 2 uoverensstemmelser pr. comparator der mangler, i dette tilfælde ved plads 1 og 2. Her repræsenterer scoren 2 derfor, at denne sub-liste mangler en comparator, for at blive en sorteret liste. Denne prune estimerer derfor hvor tæt et filter er på at blive til et comparator netværk, baseret på hvor mange comparatorer den mangler. *Second\_prune* vil dog næsten altid overestimere hvor mange comparator et filter mangler, da den ikke tager højde for at en uoverensstemmelse der kan rettes af en comparator i en sub-liste, også ville kunne fikse en uoverensstemmelse i en anden sub-liste hvis de har samme pladser.

Ved at sammenligne hvert element i et output kan variablen *count* tælle alle instanser, hvor output elementet ikke er lig med det samme som index'et i *identical\_sorted\_list*. Dermed bestemmes der hvor mange elementer i outputtet, som ikke er sorteret. *Count* ender dermed som summen af alle instanser af "fej" i hele listen af outputs. Hvorefter den tilføjes som element i listens score. Til sidst bruges score listen i *keep-percentage-filters* der skal prune filtre efter scoringssystemet er anvendt.

Ved dette eksempel:

Ikke sorteret:  $[[0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1, 1], [0, 0, 1], [1, 0, 1], [0, 1, 1], [1, 1, 1]]$

Sorteret:  $[[0, 0, 0], [0, 0, 1], [0, 0, 1], [0, 1, 1], [0, 0, 1], [0, 1, 1], [0, 1, 1], [1, 1, 1]]$

Bliver 2 outputs sammenlignet. Den første er en liste på 3 kanaler der kun har fået en comparator på. I dette tilfælde vil scoren for det filter være 4.

### 4.3 third\_prune

*third\_prune* tager udgangspunkt i endnu et scoringssystem. Med samme princip som i *second\_prune*, hvor programmet kan optimeres.

Der er flere måder at bestemme hvorvidt en liste er sorteret. Specielt i forhold til en binære liste, hvor scoren kan determineres, efter hvor 1-tallerne står i forhold til hinanden. Hvis et 1-tal står til højre, i en liste er det mere sorteret end hvis et 1-tal der står til venstre i listens placering. Dette kan måles ved brug af list comprehension.

```
[k for k in range(len(w[0].output[0])) if w[i].output[j][k] == 1] + [0]
```

Overstående viser en metode til at tælle instanser i en liste når et argument gælder. I dette tilfælde ønskes der at finde de elementer i index listen, hvor den tilsvarende plads i listen med outputs er lig med 1. Dvs. at  $[0, 1] = 1$ . Med denne tælle teknik er det muligt at finde en værdi for summen af 1-tallernes pladser. Som vist i eksemplet ovenfor lægges  $[0]$  på listen. Det gøres for at undgå en tom liste når en liste kun består af 0'er

Outputtet  $[0, 1, 0, 1]$  bruges i dette tilfælde som eksempel. Summen af index'erne må i dette tilfælde være 1 og 3, og dermed er summen lig med 4. Siden der står et 1-tal på index 1 og 3 i listen. For at holde resultatet jævnt alt efter hvor mange 1-taller der i listen, blev der valgt at dividere den samlede indeks værdi med antallet af 1-taller for hver sub-liste. Reduce funktionen kan bestemme antallet af 1-taller i listen, da det er lig med summen af elementerne i listen. I det pågældene eksempel er der 2 1-taller i listen, så scoren må være  $\frac{4}{2} = 2$ . Dette score-system kan anvendes på alle outputs i et filter og derved summeres en liste af scores, der repræsenterer "kvaliteten" af et filter. Det kunne evt. se sådan ud:

Output:  $[[0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1, 1], [0, 0, 1], [1, 0, 1], [0, 1, 1], [1, 1, 1]]$

- $[0, 0, 0] = 0$
- $[0, 0, 1] = \frac{2}{1} = 2$
- $[0, 1, 0] = \frac{1}{1} = 1$
- $[0, 1, 1] = \frac{3}{2} = 1,5$
- $[0, 0, 1] = \frac{2}{1} = 2$
- $[1, 0, 1] = \frac{2}{2} = 1$
- $[0, 1, 1] = \frac{3}{2} = 1,5$
- $[1, 1, 1] = \frac{3}{3} = 1$

Samlet score = 10

Gruppen brainstormede videre om andre scorings metoder, hvor ideen om at summere indeks værdierne kom frem:

Output:  $[[0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1, 1], [0, 0, 1], [1, 0, 1], [0, 1, 1], [1, 1, 1]]$

- $[0, 0, 0] = 0$
- $[0, 0, 1] = 2$
- $[0, 1, 0] = 1$
- $[0, 1, 1] = 3$
- $[0, 0, 1] = 2$
- $[1, 0, 1] = 2$
- $[0, 1, 1] = 3$
- $[1, 1, 1] = 3$

Samlet score = 16

Denne scoring virkede bedre da den skabte større spredning mellem scoringerne, og derved kunne gøre pruningerne endnu mere præcise. For at bygge videre på denne ide, blev der implementeret at hver indeks værdi blev sat i anden potens:

Output:  $[[0, 0, 0], [0, 0, 1], [0, 1, 0], [0, 1, 1], [0, 0, 1], [1, 0, 1], [0, 1, 1], [1, 1, 1]]$

- $[0, 0, 0] = 0$
- $[0, 0, 1] = 4$
- $[0, 1, 0] = 1$
- $[0, 1, 1] = 5$
- $[0, 0, 1] = 4$
- $[1, 0, 1] = 4$
- $[0, 1, 1] = 5$
- $[1, 1, 1] = 5$

Samlet score = 28

Her er spredningen endnu højere så gruppen besluttede at teste de forskellige metoder, hvor at summere indekserne i anden potens var hurtigere.

I *third\_prune* er et højt scoret filter dårligere end et lavt scoret filter. Derfor er det vigtigt at score gemmes som et negativt tal. Således holder funktionerne samme format og de bliver nemmere at bruge, især tænkt i forhold til hjælpefunktionerne som også bruges i *second\_prune*.



## 5 Diskussion

### 5.1 Sammenligning af køretid

Når de forskellige algoritmer skulle testes for køretid, blev det gjort på samme computer med 10 forskellige forsøg, hvorfra den bedste tid af de 10 forsøg blev udvalgt. Dette er gjort for at holde strukturen til sammenligning mellem algoritmer.

Hvis 2 algoritmer begge tager f.eks. 0,005 sekunder at finde det optimale sorteringsnetværk, argumenterer gruppen for at forskellen mellem dem er irrelevant. Det er kun hvis køretiden kommer over et par sekunder at det bliver relevant at sammenligne køretiden.

### 5.2 Scoringsystem

*second-prune* og *third-prune* bruger begge et scoringssystem, som blev udtænkt af gruppen. Formålet med dette scoringssystem er, at programmet kan veksle mellem filter baseret på hvor sorteret det er. Hvilket i dette projekt bliver gjort på 2 forskellige måder. *second-prune* er væsentligt mere effektiv. Dette skyldes metoden, som er brugt i forhold til scoring af filtrene. En særlig forskel på de to prune metoder kan eksempelvis tydes på følgende måde.

$$\begin{array}{l} \textit{second\_prune} \\ [1001] \rightarrow \textit{Score} = 2 \\ [0101] \rightarrow \textit{Score} = 2 \end{array}$$

$$\begin{array}{l} \textit{third\_prune} \\ [1001] \rightarrow \textit{Score} = -1/2 \\ [0101] \rightarrow \textit{Score} = -1 \end{array}$$

I overstående eksempel vises det tydeligt, at *second-prune* og *third-prune* scorer på forskellige måder, og dermed giver funktionerne forskellige resultatet. Dette skyldes at *second-prune* tæller uoverensstemmelser. Det betyder også at en liste med en score på 2, er 1 mulig comparator fra at være helt sorteret.

Denne logik kan ikke overføres til *third-prune*. Da den scorer efter 1 tallernes placering og veksler derfor mellem de to outputs, også selvom de begge kun mangler en comparator. *third-prune* vil derfor muligvis prune den ene af de to filtre, selvom den er lige relevante.

### 5.3 Optimering af *second-prune*

Gruppen har overvejet adskillige metoder for optimering af *second-prune*. *second-prune* approksimerer mængden af comparators et givent netværk mangler, for at blive et sorteringsnetværk. Hvilket vises et udsnip af et eksempel med output som:

|                 |                                |
|-----------------|--------------------------------|
| Filter output:  | $[[1, 0, 0, 1], [1, 1, 0, 1]]$ |
| Sorteret liste: | $[[0, 0, 1, 1], [0, 1, 1, 1]]$ |

Her kan der observeres at for at disse outputs bliver sorteret, mangler netværket *en* comparator ved kanal 0 og 2. Men filtret vil alligevel få en score på 2, da den tæller hvor mange comparators der mangler for hver sub-liste i outputs, hvorefter den lægger dem sammen. Dette leder til at approksimeringen ikke er optimal i forhold til at beregne hvor mange comparators et filter mangler. For at løse dette blev der indbygget en form for hukommelse, der gemte hvilke comparator der manglede, og dermed undgik duplikator i sub-listerne. Så hvis *second-prune* opdagede en uoverensstemmelse, blev den kanal uoverensstemmelsen var på gemt. Næste gang der så var en ny uoverensstemmelse, ville den ikke blive talt hvis den var på en af de gemte kanaler.

Her er der dog et problem der kan illustreres vis det tidligere eksempel udvides:

|                 |                                              |
|-----------------|----------------------------------------------|
| Filter output:  | $[[1, 0, 0, 1], [1, 1, 0, 1], [1, 0, 1, 0]]$ |
| Sorteret liste: | $[[0, 0, 1, 1], [0, 1, 1, 1], [0, 0, 1, 1]]$ |

Her vil den optimerede *second\_prune* gemme de kanaler med uoverensstemmelser er  $[0, 2, 3]$ , og derved vil dette filter få en score på 1, 5.

Problemet her skyldes ved uoverensstemmelser, kendes der kun en kanal comparatoren skal være på (der hvor den første uoverensstemmelse er) ikke der hvor den anden kanal skal være. Dette leder til situationer hvor der kan opstå en uoverensstemmelse mellem kanal 0 og 2 i en subliste samt 0 og 3 i den næste, men 0 vil kun repræsenteres en gang. Ved yderligere testning vidste det sig, at denne metode er langsommere end den originale *second\_prune*.

Dette skyldes, at ingen af metoderne 100% repræsenterer den manglende mængde comparators. Men siden den optimerede *second\_prune* ikke tæller duplikater, bliver disse forskelle mindre markante, i forhold til den den øgede score interval i den originale *second\_prune*. Når score forskellene er mere markante, vil selv små forskelle være nemmere at måle, og derved vil det lede til bedre resultater når prune f.eks. kun tager de 15% bedste. Dette har gjort at gruppen har vurderet at dette ikke er en relevant optimering af *second\_prune* og derfor ikke er implementeret.

## 5.4 Kombinerer af prunes

Der er blevet udforsket metoder til at kombinere de forskellige prune funktioner, for at optimere networkFinder. Dette gøres, i teori, ved at kombinere prune funktionerne for at fjerne flest mulige filtre, hvor det vil optimere programmets køretid, samtidig med at der stadig bliver fundet optimale sorteringsnetværk.

Muligheden for at kombinere projektets *second-* og *third\_prune* er muligt, ved at lægge de 2 scores sammen, hvor der derved findes de bedste filtrer. Men som tidligere argumenter viser, menes der at *second\_prune* er bedre til at vurdere filtrer end *third\_prune*. Dermed ville det ikke være relevant, at inddrage noget der er mindre nøjagtigt. Det ville, potentielt, lede til at et dårligere filter, kunne få en bedre score end det burde.

At kombinere *first\_prune* med *second* eller *third* prune giver, i teori, en del mere mening. Her ville *first\_prune*, som fjerner dubletter, blive brugt først. Hvorefter de resterende filtre bliver scoret af enten *second-* og *third\_prune*. Derved ville der kun stilles scores til færre filtre, og det ville lede til en overordnet optimering af programmet. Dette viste sig dog ikke til at være tilfældet. Det kræver f.eks. stadigvæk den samme mængde beholdte filtrer, for at finde et optimalt sorteringsnetværk på 6 kanaler, og køretiden er uændret. Selv ved flere kanaler hvor køretiden er meget højere, har det enten ingen eller en negativ effekt. Dette kunne potentielt grundet, den måde der bliver pruned, hvor der bliver beholdt de  $x$  bedste kanaler, så det er irrelevant hvor mange "dårlige" dubletter der er, da de alle sammen bliver sorteret fra. Det kunne også indikere, at tiden som det tager at tjekke om filtrene er dubletter, er højere end bare at score dem alle sammen og sortere de dårligste fra. Dette indikerer en generel tendens om optimering et sted, ikke nødvendigvis leder til en overordnet optimering for hele programmet. Derigennem blev der vurderet, at prune funktioner fungerer bedre i hvert deres sammenhæng uden forbindelse mellem hinanden.

Derfor ved at tage udgangspunkt i hvor lang tid det tager programmet at finde et optimalt sorteringsnetværk, vil lede til det bedste resultat.

## 5.5 Hvor meget skal prunes?

I forskel til *first\_prune* som grundet dens opbygning fjerner en vis mængde filtre, fjerner second -og third prune ikke direkte noget. De tildeler en score til de forskellige filtre baseret på hvor tæt på et sorteringsnetværk de er. Der kan siges ikke noget om, hvor mange der skal beholdes eller fjernes. Der skal derfor tages en beslutning bagefter, om hvor mange der skal beholdes. Dette har ret stor betydning, da hvis der fjernes for mange, vil algoritmen ikke finde et optimalt sorteringsnetværk. I modsætning hvis der fjernes for lidt, er algoritmen ikke optimalt i forhold til køretiden. For mindre kanaler er det ret relativt simpelt, at finde ud af hvor mange filtre der skal beholdes, for at finde det optimale sorteringsnetværk. Kan der bruges "brute force" indtil prune på et vis antal elementer returnerer et sorteringsnetværk med det korrekte antal comparators. Dette er ud fra antagelsen om, at den optimale mængde comparators kendes. Hvis det ikke kendes, ville det være svært at argumentere for brugen af f.eks. *keep\_x\_filters*, da den altid kan sættes til en højere værdi, for potentielt at udlede et mere optimalt sorteringsnetværk. Der ville *keep\_percentage\_filters* være mere optimal da procenten følger skaleringen af filtre når kanalantallet stiger. Samtidig med at den, f.eks. kan sættes til 100, her ville det svare til slet ikke at prune noget. *prune\_below* var gruppens første udkast til prune. *prune\_below* har den fordel at den er hurtigere end hvis *keep\_percentage\_filters*, hvis den bliver sat til 50, dog kan den ikke justeres.

Andre metoder har også været testet i forhold til at optimere pruningen. Da mængden af muligheder udfolder sig som et grafisk træ, ville det potentielt give bedre resultater, hvis træet blev holdt "bredt" i toppen, og først begyndte at indskrænke senere hen. Her er det vigtigt, at det igen ikke handler om at prune mindst muligt, men om at få den bedste køretid for algoritmen. Ved test på 5 og 6 kanaler giver det følgende udfald:

Uden brug af bredere træ i starten:

5 kanaler: 0.0748

6 kanaler: 6.0887

Ved at vente efter 1 extend med at prune:

5 kanaler: 0.5981

6 kanaler: 139.6389

Ved at vente efter 2 extends med at prune:

5 kanaler: 3.3790

6 kanaler: 1502.7319

Dette kunne indikere, at det enten ikke er en effektiv vej at gå, eller at det først er mere effektivt ved højere kanallængder. Det ville give mening i forhold til at jo højere kanallængde, jo mere eksponentielt udvikler antallet af filtre sig. Så derfor kunne det give god mening at skabe flere filtre i starten for at få bred spredning, så senere give en hårdere restriktion på hvor mange filtre der må være efter prune.

Gruppen har testet adskillige mulige måde at bruge de forskellige prune-funktioner og estimeret de hurtigste og mest optimale måde at finde sorteringsnetværker hvor  $n = 4..7$  kanaler.

Herunder er det blevet testet *second\_prune* samt *third\_prune* med alle kombinationer af hjælpefunktioner også. Her blev *third\_prune* en smule mere optimal i de mindre tilfælde, hvor  $n$  er 4 eller under, her sammen med *keep\_percentage\_filters*. Dog ved  $n$  er lig 5 eller derover. Blev kombination mellem *second\_prune* og *keep\_percentage\_filters* den mest optimale.

Her blev der også testet hvilken procentdel, det skal aftages for at skabe dette optimale sorteringsnetværk. Som ses under i tilfældet under, afhænger det af hvor stort  $n$  er.

$n = 4$  : *third\_prune* : *keep\_percentage\_filters* : 0.0090 seconds, keep 11%, size : 5 (OPT)

$n = 5$  : *second\_prune* : *keep\_percentage\_filters* : 0.0352 seconds, keep 17%, size : 9 (OPT)

$n = 6$  : *second\_prune* : *keep\_percentage\_filters* : 2.5808 seconds, keep 15%, size : 12 (OPT)

$n = 7$  : *second\_prune* : *keep\_percentage\_filters* : 56.0036 seconds, keep 10%, size : 16 (OPT)

## 6 Konklusion

Gruppen har, under projektet, udarbejdet et program som skaber det mest optimale sorteringsnetværk muligt med *second\_prune*, og implementeret 2 andre funktioner, *first\_prune* og *third\_prune*.

Disse funktioner er blevet optimeret igennem hele forløbet, for at finde den bedst mulig løsning. Under dette forløbet blev mange andre muligheder udforsket, blandet andet også kombination af flere prunes. Derudover er der også blevet vurderet hvilken kombination med hjælpefunktioner der er bedst. Vi har, igennem timestamps, vurderet kombination af *second\_prune* og *keep\_percentage\_filters*, hvor dette sættes til 17 procent for at få den mest optimale networkFinder.

## 7 Bilag

### 7.1 prune.py

```
from filter import Filter
from functools import reduce

def prune(w: list[Filter], n: int) -> list[Filter]:
    """Removes some filters on n channels from w."""
    return second_prune(w,n)

def first_prune(w:list[Filter], n:int) -> list[Filter]:
    """Returns a list of filters without duplicates of outputs"""
    i = 1
    result = [w[0]]
    while i < len(w):
        duplicate = False
        j = 0
        while j < len(result):
            if w[i].output == result[j].output:
                duplicate = True
                j = j + len(result)
            j = j + 1
        if duplicate == False:
            result = result + [w[i]]
        i = i + 1
    return result

def second_prune(w:list[Filter],n: int)-> list[Filter]:
    """Scores filters in list w and returns a new list with on n channels from w"""
    i = 0
    score = []
    while i < len(w):
        sorted_list = identical_sorted_list(w[i].output,n)
        j = 0
        count = 0
        while j < len(w[0].output):
            k = 0
            while k < n:
                if w[i].output[j][k] != sorted_list[j][k]:
                    count = count + 1
                k = k + 1
            j = j + 1
        score = score + [int(count/2)]
        i = i + 1
    return keep_percentage_filters(w,score,17)
```

```

def third_prune(w:list[Filter], n: int) -> list:
    """Prunes filters in a list (w) of filters
    on n channels by giving all filters a score based on perfomance,
    and removing all filters under with a score less than the average.
    """
    score = []
    index = range(len(w[0].output[0]))
    i = 0
    while i < len(w):
        j = 0
        count = 0
        while j < len(w[0].output):
            count = count - reduce(lambda x, y: x + (y**2),
                                   ([k for k in index if w[i].output[j][k] == 1] + [0]))
            j = j + 1
        score = score + [count]
        i = i + 1
    return keep_percentage_filters(w,score,17)

def prune_below(w: list[Filter], s:list) -> list[Filter]:
    """Prunes filters below a the average score difined in the prune fuction"""
    avg = (reduce(lambda x,y: x + y, s))/len(s)
    result = []
    i = 0
    while i < len(s):
        if s[i] <= avg:
            result = result + [w[i]]
        i = i + 1
    return result

def keep_x_filters(w: list[Filter], s:list, x: int) -> list[Filter]:
    """returns the n best filters"""
    w1 = w
    sort_by_score(w1,s)
    if x < len(w1):
        return w1[0:x]
    else:
        return w1

def keep_percentage_filters(w: list[Filter], s:list, n: int) -> list[Filter]:
    """returns the n percentage best filters"""
    w1 = w
    sort_by_score(w1,s)
    percentage = int(len(s) / 100 * n)
    if 0 < percentage:
        return w1[0:percentage]
    else:
        return w1

```

```

def sort_by_score(w: list[Filter], s: list):
    """Sorts a list of filters and list of scores with lowest scores first"""
    i = 0
    while i < len(s)-1:
        if s[i] > s[i+1]:
            s[i], s[i+1] = s[i+1], s[i]
            w[i], w[i+1] = w[i+1], w[i]
            if i != 0:
                i = i - 1
        else:
            i = i + 1

def identical_sorted_list(w: list[list[int]], n: int) -> list[list[int]]:
    """Returns an identical sorted list from filters"""
    i = 0
    w1 = []
    while i < len(w):
        sum = reduce(lambda x, y: x + y, w[i])
        w1 = w1 + [(n - sum)*[0] + sum*[1]]
        i = i + 1
    return w1

```

## Litteratur

- [1] Cruz-Filipe, L. & Codish, M. & Ehlers, T. & Müller, M. & Schneider-Kamp, P. d. *Sorting Networks: to the End and Back Again*, Table 1 (2015).

## 8 corrections for slides

```
def identical_sorted_list(w:list[list[int]],n:int)-> list[list[int]]:
    """Returns an identical sorted list from filters"""
    new_list = []
    for i in range(len(w)):
        ones = sum(w[i])
        new_list = new_list + [(n - ones)*[0] + ones*[1]]
    return new_list

def prune(w:list[Filter], n:int) -> list[Filter]:
    """Removes some filters on n channels from w"""
    return second_prune(first_prune(w),n)
```